

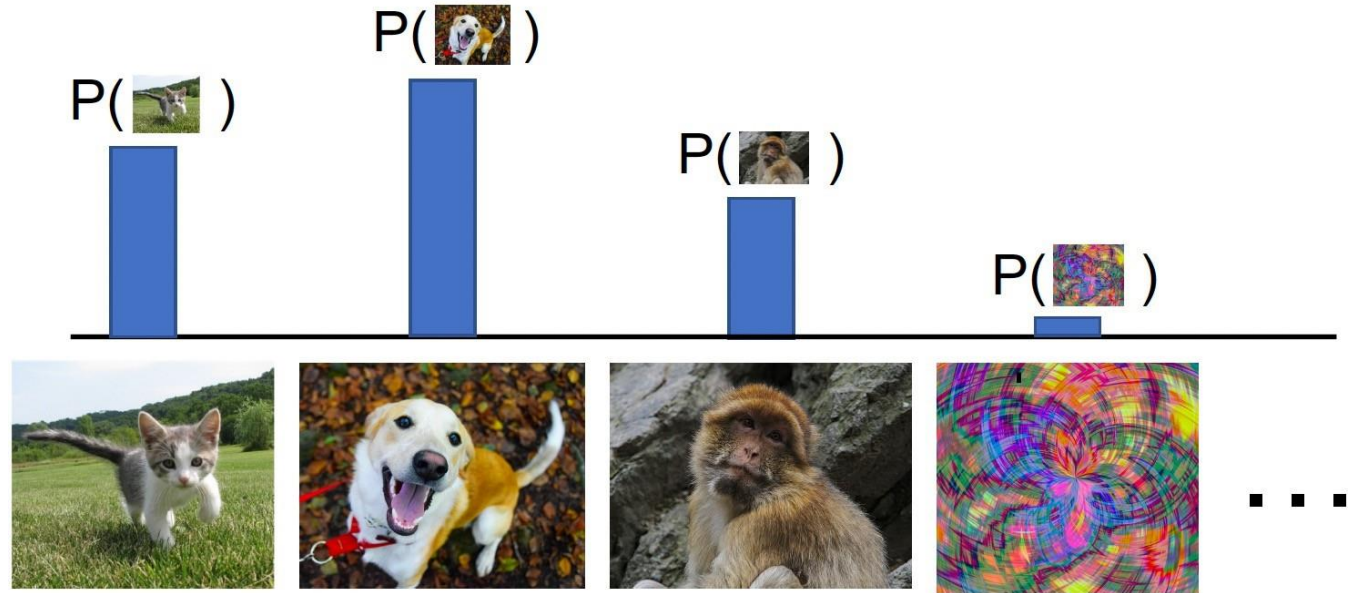
Discriminative vs Generative models

Discriminative Model:

Learn a probability distribution $p(y|x)$

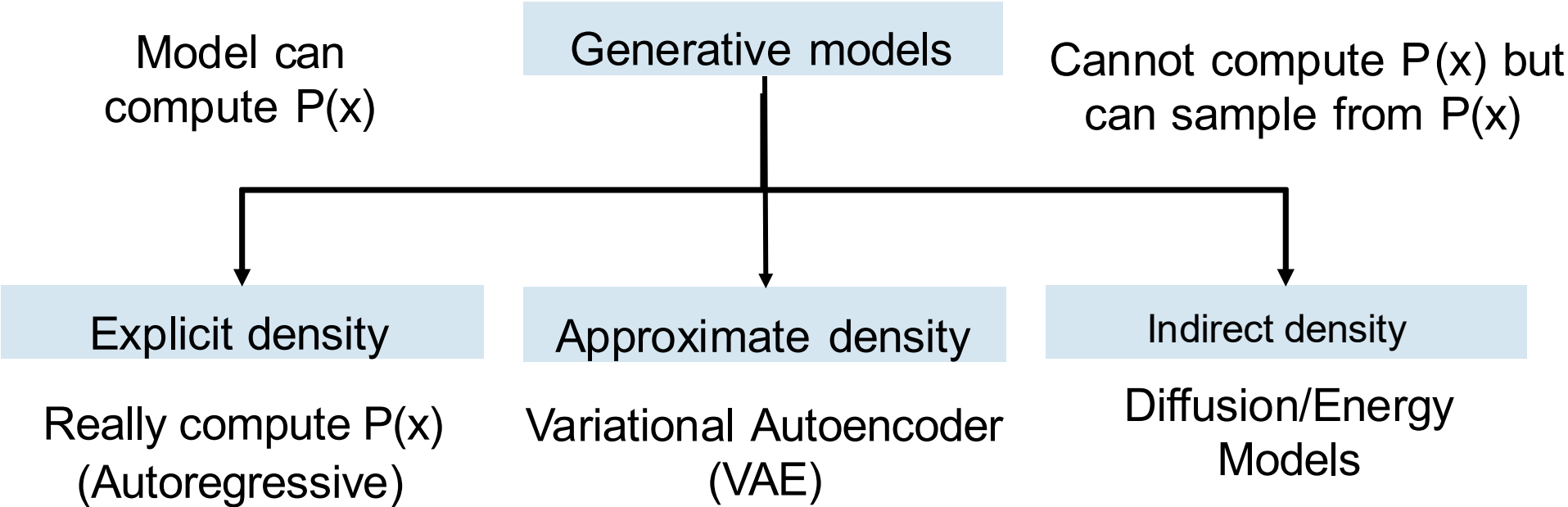
Generative Model:

Learn a probability distribution $p(x)$



- Generative model: All possible images compete with each other for probability mass
- Model can “reject” unreasonable inputs by assigning them small values

Taxonomy of Generative Models

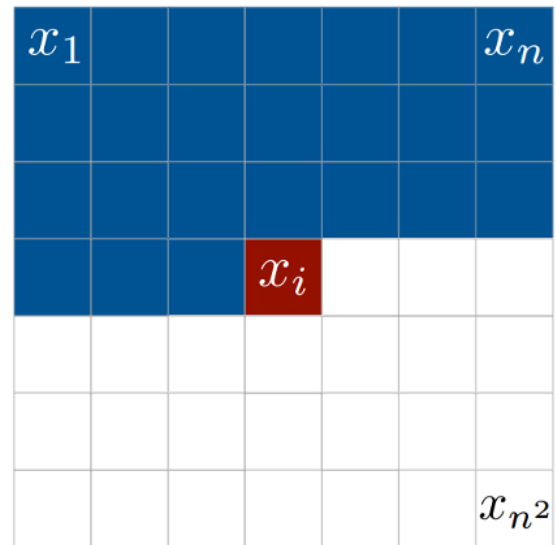


Autoregressive network

Use chain rule to decompose likelihood of an image x into product of 1-d distributions:

$$p(x) = p(x_1, x_2, \dots, x_{n^2}) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \dots$$

Express using a sequence neural network!



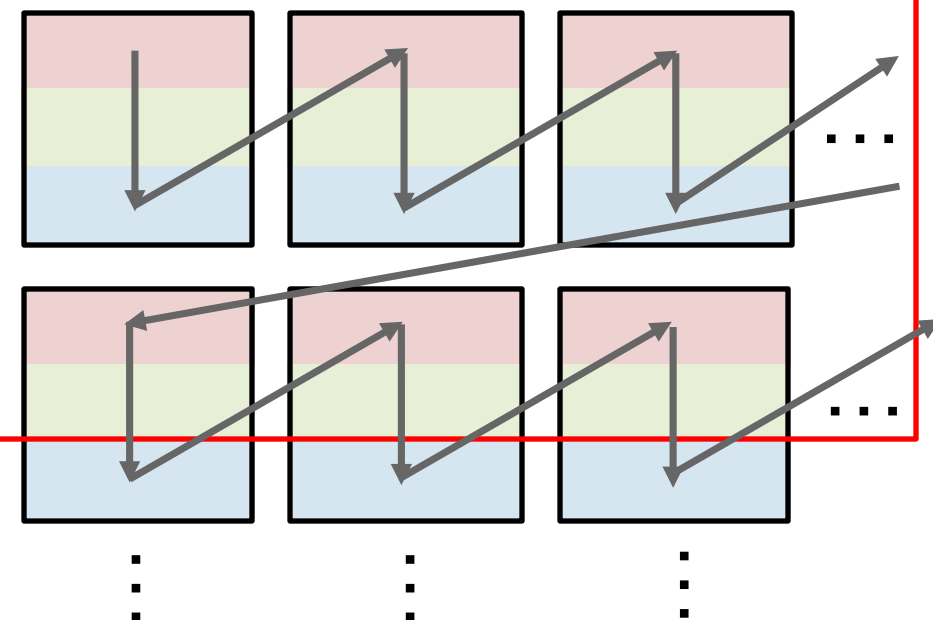
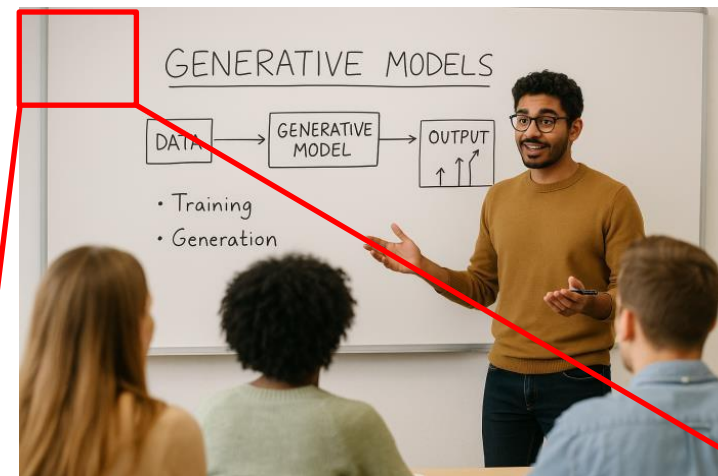
Autoregressive Models of Images

Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values $[0 \dots 255]$

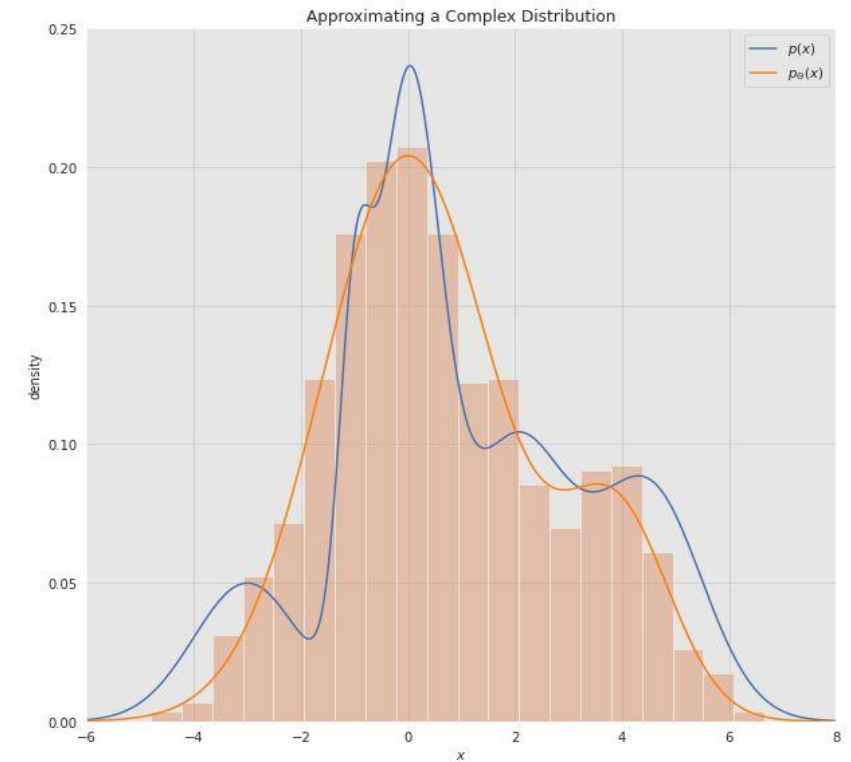
Model with an RNN or Transformer

Problem: Too expensive. 1024×1024 image is a sequence of 3M subpixels

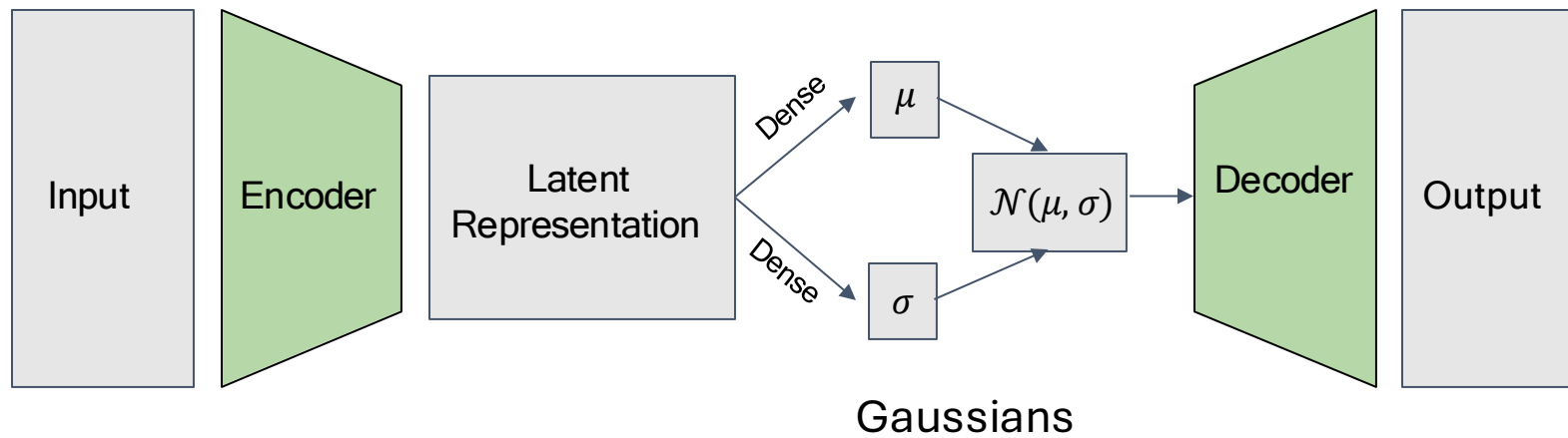


Generative Modeling with Gaussians

- The distribution $p(x)$ is **complex**
- We want to work with **simple** distributions (such as Gaussians)
- **Idea:** learn a NN to transform a simple distribution sample into a complex one

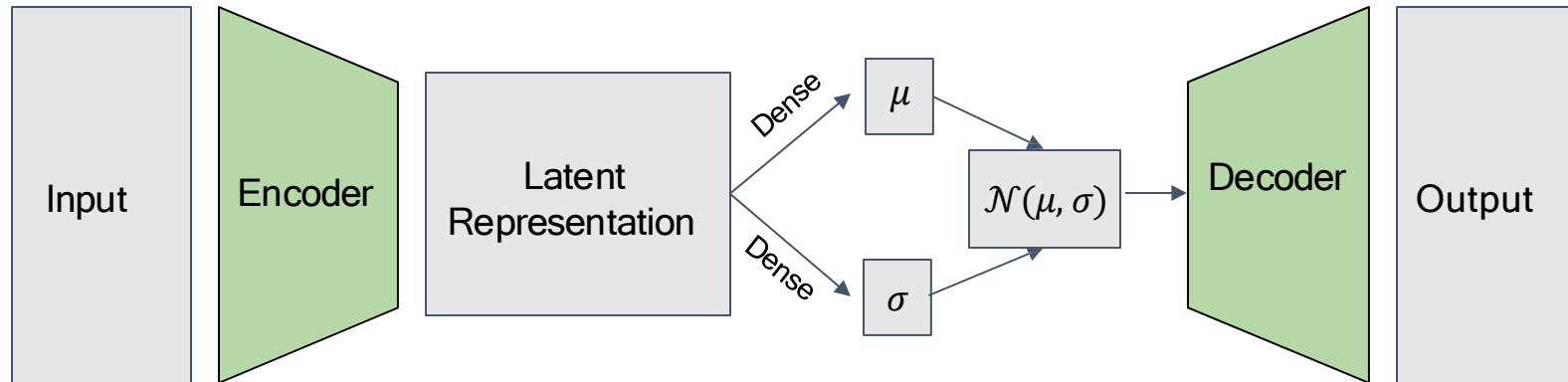


Variational Auto-encoder

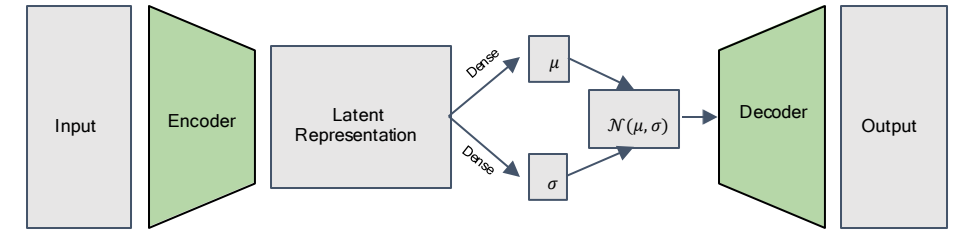


Putting it all together

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$



One more practical detail



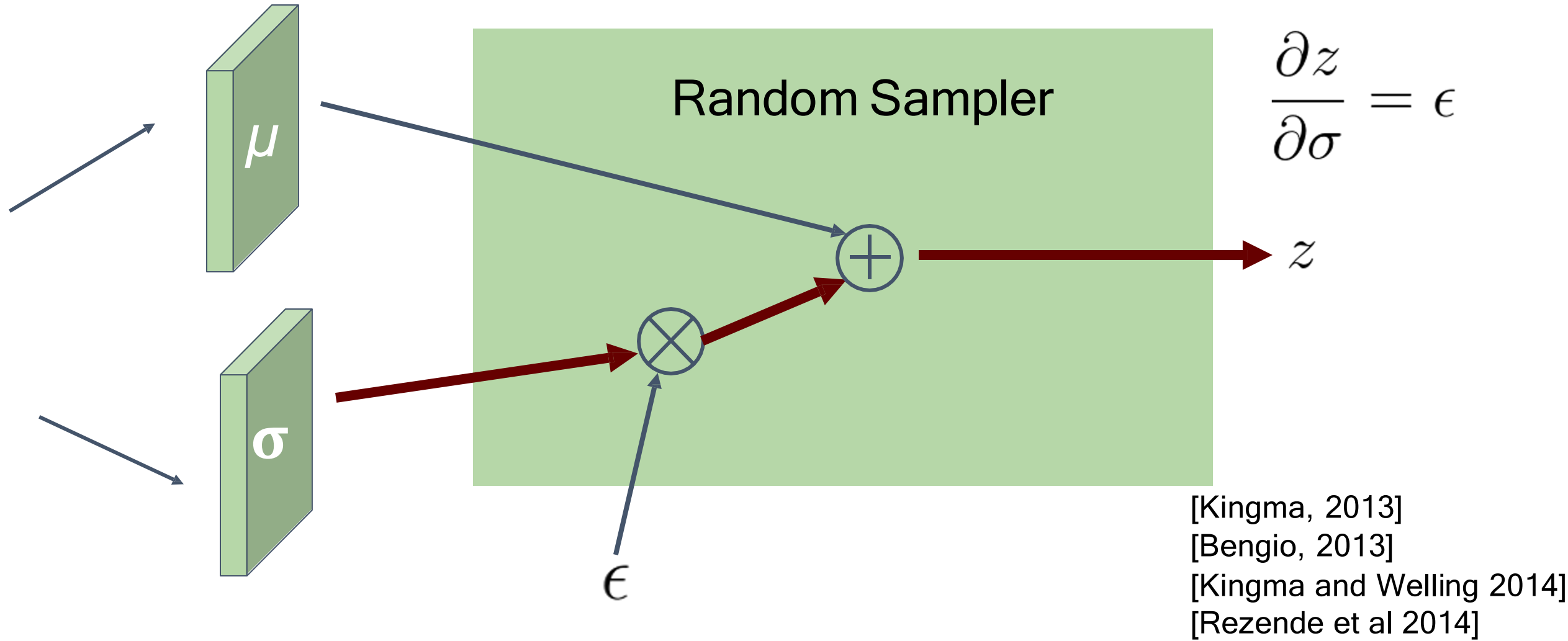
Let's again consider our sampling operation

$$z \sim \mathcal{N}(\mu, \sigma)$$

$\mu_i \in [-\infty, \infty]$ $\sigma_i \in [0, \infty]$

- Nothing prevents the neural network from outputting ***negative*** values for the standard deviation.
- Instead of predicting σ , we will instead predict $\log(\sigma^2)$. This ensures that every $\sigma_i \in [0, \infty]$
 - i.e. just treat the output of the Dense layer as if it is $\log(\sigma^2)$

Random Sampler with Reparameterization Trick



KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\sigma = 1$

$$\begin{aligned} D_{KL}(\mathcal{N}(\mu, 1) || \mathcal{N}(0, 1)) &= \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + 1^2 - \ln(1) - 1) \\ &= \frac{1}{2} \sum_{i=1}^k \mu_i^2 \end{aligned}$$

The expression is minimized $\mu = 0$ (which is what we want)

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\mu = 0$

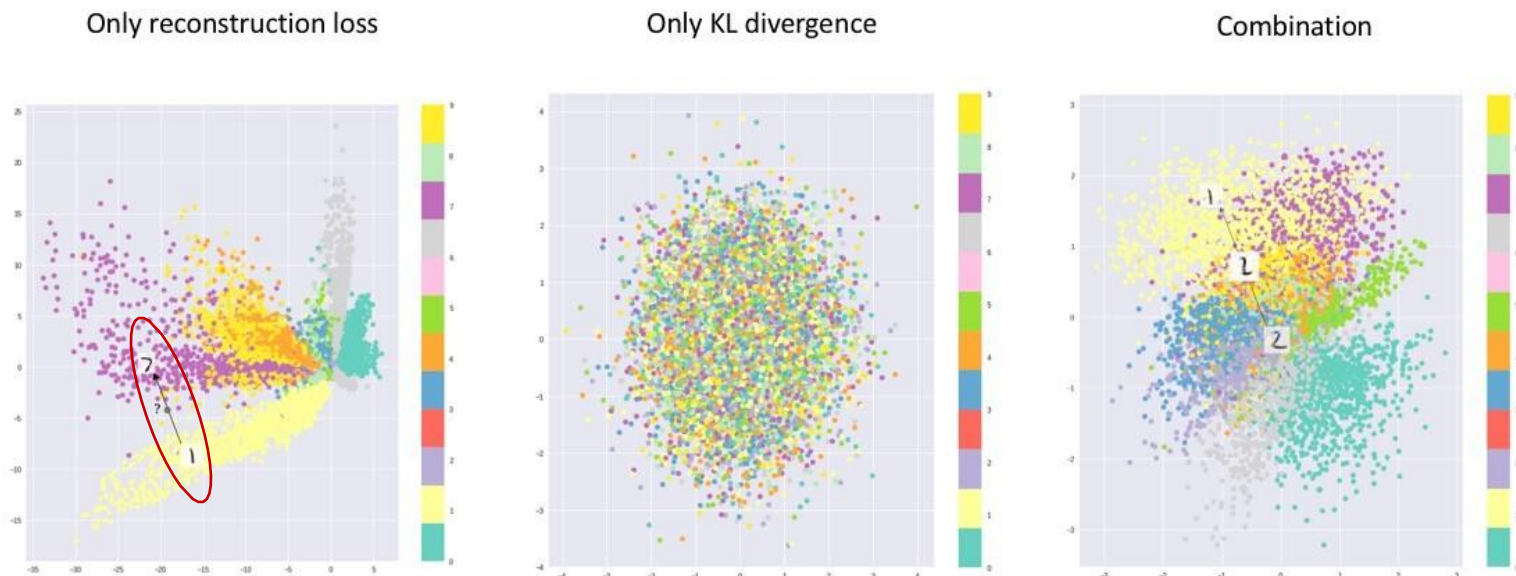
$$D_{KL}(\mathcal{N}(0, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 - \ln(\sigma_i^2) - 1)$$

This expression is minimized when $\sigma = 1$ (which is also what we want)

Latent Space Interpolation

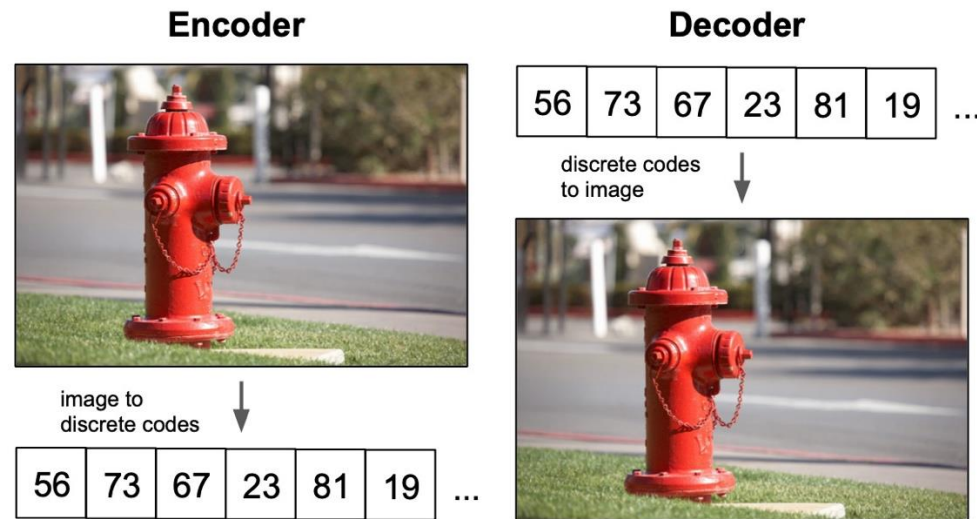
- Can also try it with a regular autoencoder
 - Doesn't work as well
 - ***Why not?***
 - The KL divergence loss regularizes the shape of the latent space. Without it, a regular autoencoder might have “empty” pockets of latent space

Linear interpolation has to cross a pocket of empty space (where the behavior of the decoder is not well defined)

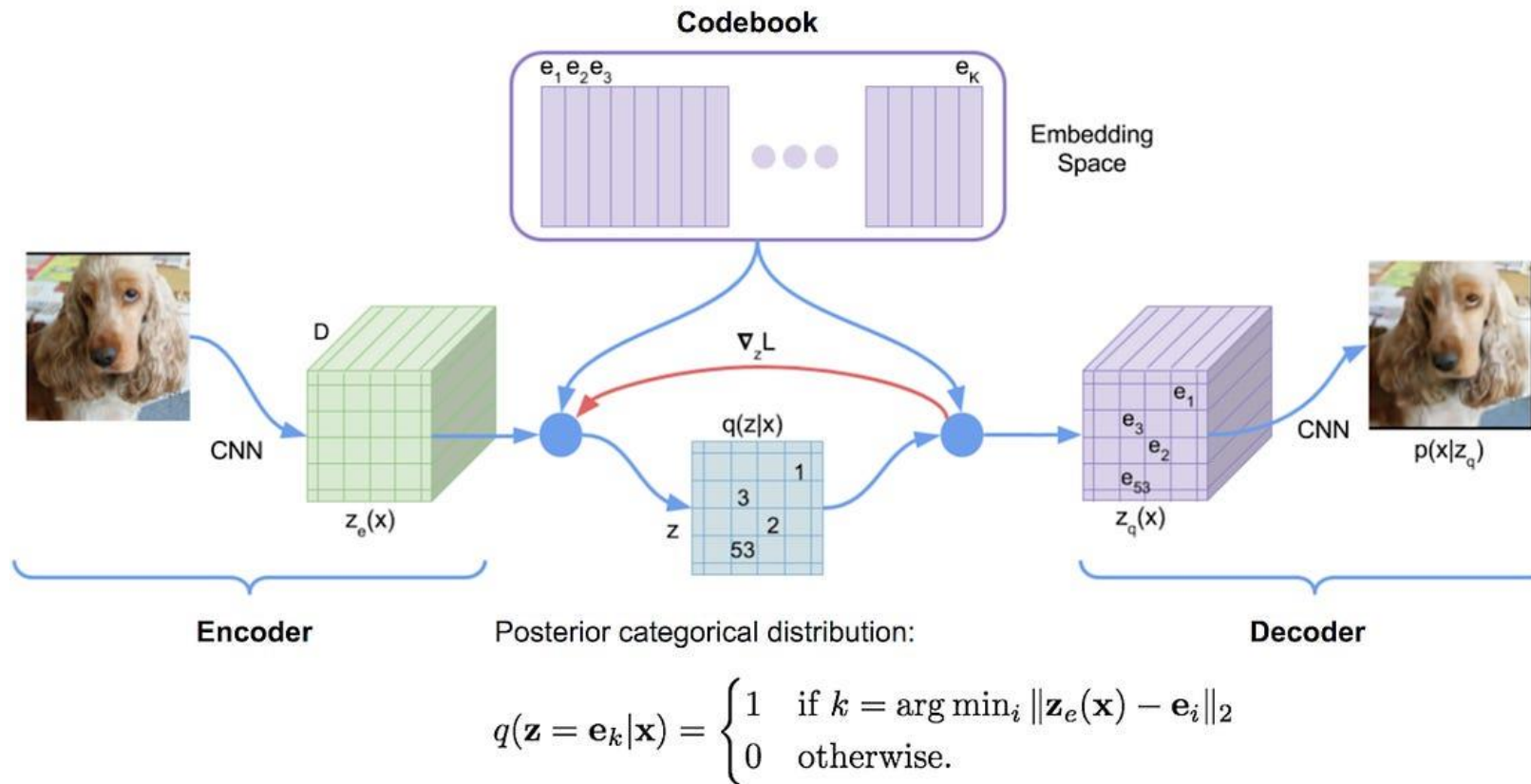


Vector Quantized Variational Autoencoder (VQ-VAE)

- So far, we have seen how continuous vector spaces can be used to represent the latents in an autoencoder.
- But latents do not necessarily need to be continuous vectors



VQ-VAE training



Pick a Codebook index by taking the closest codebook vector
The min operation require tricks to backprop